Lecture 3 - Dart OOP II + Async

Abstraction, Encapsulation, Mixins, Generics + Future/await

Fettah KIRAN

Department of AI and Data Engineering

Yıldız Technical University

fkiran@yildiz.edu.tr

Overview

1. Recap Quiz — Week 2 (constructors, inheritance, polymorphism, null-safety)
2. Encapsulation: `_private`, getters/setters, immutability
3. Abstraction: `abstract class Solver` — extends vs implements vs with
4. Mixins + Extensions: `Validatable`, `Loggable`, `NumericalMixin`
5. Generics: `Box<T>`, `Solver<T>`, `Matrix<T extends num>`
6. Collections + Exceptions + Equality in 10 minutes
7. Async: `Future`, `async/await`, `Future.wait`, errors, queues, Streams preview, Isolates
8. Lab: abstract `Shape`, interfaces, multiple interfaces, `ElectricCar` challenge

Encapsulation — Why Hide State?

- **Dart rule:** leading `_` = library-private. No `private` keyword.
- Protect invariants: `tolerance > 0`, `maxIter >= 1`, matrix dims fixed
- Expose intent via getters/setters, keep fields immutable where possible
- Math example: nobody should set `iterations = -5` mid-solve

1 — Private Fields (_)

```
class BisectionSolver {
  final double Function(double) f;
  double _tolerance = 1e-6;   // private
  int _maxIter = 100;         // private

  BisectionSolver(this.f);
  int get iterations => _iterCount;
  int _iterCount = 0;
}
```

- Same file can see `_tolerance` (library scope!) — split files to enforce
- `final f` = set once, never reassigned; `_iterCount` mutated internally only

2 — Getters & Setters (Validated)

```
double get tolerance => _tolerance;
set tolerance(double v) {
  if (v <= 0 || v >= 1) throw ArgumentError('tol in (0,1)');
  _tolerance = v;
}
// computed getter — no backing field
bool get converged => _residual < _tolerance;
```

- Getter/setter look like fields to callers: `s.tolerance = 1e-8;`
- Validate once here → every solver stays numerically sane

3 — Immutable Config with copyWith

```
class SolverConfig {
  final double tolerance;
  final int maxIter;
  const SolverConfig({this.tolerance = 1e-6, this.maxIter = 100});

  SolverConfig copyWith({double? tolerance, int? maxIter}) =>
    SolverConfig(
      tolerance: tolerance ?? this.tolerance,
      maxIter: maxIter ?? this.maxIter,
    );
}
```

- `final` = per-instance once; `const` = compile-time canonical
- Flutter pattern too: `ThemeData.copyWith(...)` — same idea

Abstraction — Contracts, Not Code

- `abstract class` = cannot be instantiated, defines **what** not **how**
- Lets different shape types expose the same required API
- Core lab contract (abstract shape):

```
abstract class Shape {
  double area();
  double perimeter();
}
```

- `Rectangle` and `Circle` implement the shape behavior in the lab

Abstract Solver in Action

```
abstract class Solver<T> {
  T solve();                          // contract
  void validate() {}                  // optional hook
}

class BisectionSolver extends Solver<double> {
  final double Function(double) f;
  final double a, b;
  BisectionSolver(this.f, this.a, this.b);
  @override double solve() {
    // ... bisection loop, return midpoint
    return 1.4142;
  }
}

Solver<double> s = BisectionSolver((x) => x * x - 2, 0, 2);
print(s.solve()); // 1.4142 — sqrt(2)
```

extends vs implements vs with

Keyword	Meaning	Use when
`extends`	IS-A, one parent, inherits code	`BisectionSolver extends Solver<double>`
`implements`	Contract only, re-write everything	Mock solver in tests
`with`	Mixin reuse, no hierarchy	`with ConvergenceLogger`

- Dart: single inheritance + unlimited `with` / `implements`
- Rule of thumb: `extends` for taxonomy, `with` for abilities, `implements` for fakes

Mixins — Reusable Abilities

- `mixin` = bundle of methods/fields, no constructor, plugged with `with`
- Keeps hierarchy flat: logging ≠ validation ≠ solving
- Stack them: `class S extends Solver<double> with Loggable, Validatable`

```
mixin Loggable {
  void log(String m) => print('[LOG] $m');
}
mixin Validatable {
  void check(bool c, String m) {
    if (!c) throw StateError(m);
  }
}
```

Mixins for Numerics

```
mixin ConvergenceLogger {
  final List<double> history = [];
  void track(double residual) {
    history.add(residual);
    if (history.length % 10 == 0) print('iter ${history.length}: $residual');
  }
}

class BisectionSolver extends Solver<double> with ConvergenceLogger {
  @override double solve() {
    double r = 1.0;
    track(r); // logs convergence
    return r;
  }
}
```

- `history` powers Week 10 charts (fl_chart) — collect now, plot later
- Same mixin reused by Newton + Simpson, zero duplication

Extensions — Add Methods Without Touching the Class

```
extension DoubleFormat on double {
  String get fixed6 => toStringAsFixed(6);
}
extension ResidualList on List<double> {
  bool get isMonotone =>
    every((_) => true) && _ok();
  bool _ok() {
    for (int i = 1; i < length; i++) {
      if (this[i] > this[i-1]) return false;
    }
    return true;
  }
}

print(1.41421356.fixed6); // 1.414214
```

- Great for math formatting, unit conversion, list stats
- Cannot add state — methods/getters only; import to use

Generics — One Code, Many Types

```
class Box<T> {
  final T value;
  Box(this.value);
  T get unbox => value;
}
var a = Box<double>(1.41);
var b = Box<Matrix>(Matrix(2, 2)); // same class!
```

- `T` resolved at compile time → type-safe, no casts
- Inference usually works: `Box(42)` = `Box<int>`

Generic Solver + Constraints

```
abstract class Solver<T> {
  T solve(); // double root, Vector system, Matrix inverse...
}
class NewtonSolver extends Solver<double> { ... }
class LinearSolver extends Solver<Vector> { ... }
```

Constraints keep math valid:

```
T add<T extends num>(T x, T y) => (x + y) as T;
```

- `T extends num` → only `int` / `double` allowed; `+` guaranteed
- Generic methods: `<T>(...)` on functions, not just classes

Generic Matrix<T extends num>

```
class Matrix<T extends num> {
  final List<List<T>> data;
  Matrix(this.data);
  int get rows => data.length;
  int get cols => data.first.length;

  Matrix<double> operator +(Matrix<double> o) {
    // element-wise add -> doubles
    return Matrix(List.generate(rows,
      (i) => List.generate(cols, (j) => data[i][j] + o.data[i][j])));
  }
}
var m = Matrix<double>([[1.0, 2.0], [3.0, 4.0]]);
```

- Week 2 `Matrix` reborn type-safe: `Matrix<int>` vs `Matrix<double>`
- Constraint blocks `Matrix<String>` at compile time

Collections — List / Map / Set

- `List<double> residuals = []` — ordered history
- `Map<String, double> params = {'tol': 1e-6}` — named config
- `Set<int> visitedPivots = {}` — dedupe, O(1) lookup
- Prefer `final` list (mutable contents, fixed reference) vs `const` (frozen)

Spread + Collection-if/for

```
var base = [1.0, 2.0];
var full = [...base, 3.0, ...?maybeNull]; // spread + null-aware
var flags = {'verbose': true, if (debug) 'trace': true}; // collection-if
var squares = [for (var x in [1, 2, 3]) x * x];           // collection-for
var grid = [for (int i = 0; i < 3; i++) for (int j = 0; j < 3; j++) i * j];
```

- Builds UI lists + numeric grids with zero loops
- `...?` skips null safely — perfect for optional history series

Exceptions — try / catch / finally

```
try {
  solver.tolerance = -1;       // throws ArgumentError
  final r = solver.solve();
  print(r.fixed6);
} on ArgumentError catch (e) {
  print('Bad param: $e');
} catch (e) {
  print('Solve failed: $e');
} finally {
  print('iters: ${solver.iterations}');
}
```

- `on Type` = specific, `catch` = general, `finally` = always
- Never swallow silently in numerics — log residual + bounds

Custom Exceptions (Math-Grade Errors)

```
class NoBracketException implements Exception {
  final String msg; NoBracketException(this.msg);
  @override String toString() => 'NoBracket: $msg';
}
void check(bool ok, String m) {
  if (!ok) throw NoBracketException(m);
}
// usage: check(f(a) * f(b) < 0, 'f(a),f(b) same sign');
```

- `implements Exception` = pure contract, you write `toString`
- Callers catch `on NoBracketException` separately from network errors

Equality & hashCode

- Default `==` = identity (same object). Math needs **value** equality.
- Override both together — `Set` / `Map` keys depend on `hashCode`

```
class Frac {
  final int n, d; Frac(this.n, this.d);
  @override bool operator ==(Object o) =>
    o is Frac && n * o.d == o.n * d;
  @override int get hashCode => Object.hash(n / d, d);
}
```

- Rule: equal objects → equal hashCodes. Use `Object.hash(...)`.
- Packages `equatable` (later) removes boilerplate

Async — Why Mobile Can't Block

- UI runs on **one thread**: blocking 2 s solve = frozen 120 frames
- Solution: `Future` (one later value) + `async/await` (readable waits)
- Heavy solve → `await` it, show spinner; result rebuilds via `setState` / `FutureBuilder` (W4-5)
- Today: console-level async. UI wiring comes Weeks 4-5 + 9.

Future States

- **Uncompleted** → pending (solver running, HTTP flying)
- **Completed with value** → `Future<double>.value(1.4142)`
- **Completed with error** → `Future.error(NoBracketException(...))`
- Chain: `.then()`, `.catchError()`, `.whenComplete()` — or cleaner `async/await`

```
Future ──→ value ✅   (then / await)
       └──→ error ❌   (catchError / try-catch)
```

async / await

```dart
Future<double> solveAsync(Solver<double> s) async {
  await Future.delayed(Duration(milliseconds: 300)); // yield UI
  return s.solve(); // runs, returns later
}

void main() async {
  print('solving...');
  double r = await solveAsync(BisectionSolver((x) => x * x - 2, 0, 2));
  print('root = ${r.toStringAsFixed(6)}');
}
```

- `async` returns `Future`; `await` pauses **this** function only, UI stays alive
- `main() async` allowed in Dart — top-level await in scripts

Parallel: Future.wait

```
Future<void> compare() async {
  var results = await Future.wait([
    Future(() => BisectionSolver((x) => x * x - 2, 0, 2).solve()),
    Future(() => NewtonSolver((x) => x * x - 2, (x) => 2 * x, 1).solve()),
  ]);
  print('bisection=${results[0]}, newton=${results[1]}');
}
```

- All futures run concurrently; list completes when **all** complete
- One failure fails the whole `wait` — wrap with `try/catch` or `Future.wait(..., eagerError: false)`

Async Error Handling

```
try {
  double r = await solveAsync(badSolver);
  print(r);
} on NoBracketException catch (e) {
  print('Fix bracket: $e');
} catch (e) {
  print('Other: $e');
} finally {
  hideSpinner();
}
```

- Same `try/catch` shape, just `await` inside
- `.catchError()` alternative — `try/catch` preferred for readability
- Always clear loading state in `finally`

Microtask vs Event Queue (Brief)

- **Microtasks:** urgent, run before next event (`scheduleMicrotask`, completed futures)
- **Event queue:** timers, I/O, gestures, `Future.delayed`, isolates messages
- Loop: drain microtasks → one event → drain microtasks → ...

> Mental model: microtask = VIP lane, event = normal lane. Don't flood VIP lane or UI starves.

Stream Preview (→ Week 9)

- `Future<T>` = **one** value later. `Stream<T>` = **many** values over time.
- Solver streaming residuals → live convergence chart:

```
Stream<double> residuals() async* {
  yield 1.0; yield 0.25; yield 0.03; // await + yield in real loop
}
```

- Week 9: `StreamBuilder` + Firestore live notes. Same pattern, cloud source.

Isolates — True Parallelism (1 Slide)

- Dart is single-threaded per isolate; heavy CPU ($10^6 \times 10^6$ matrix) still janks even with `async`
- `Isolate.run(() => bigSolve())` = separate memory, true core parallelism
- Cost: no shared memory — message-passing only, serialization overhead
- Rule: `async` for waiting (I/O), **isolate** for crunching. Lab stays single-isolate; W12 revisits.

Math Solver 1 — Bisection

```
class BisectionSolver extends Solver<double> with ConvergenceLogger {
  final double Function(double) f;
  final double a, b;
  final SolverConfig cfg;
  BisectionSolver(this.f, this.a, this.b, [this.cfg = const SolverConfig()]);
  @override double solve() {
    double lo = a, hi = b, mid = lo;
    for (int i = 0; i < cfg.maxIter; i++) {
      mid = (lo + hi) / 2;
      double r = f(mid).abs(); track(r);
      if (r < cfg.tolerance) return mid;
      if (f(lo) * f(mid) < 0) hi = mid; else lo = mid;
    }
    return mid;
  }
}
// BisectionSolver((x) => x*x-2, 0, 2).solve() ≈ 1.414214
```

Math Solver 2 — Newton

```
class NewtonSolver extends Solver<double> with ConvergenceLogger {
  final double Function(double) f, df;
  final double x0; final SolverConfig cfg;
  NewtonSolver(this.f, this.df, this.x0, [this.cfg = const SolverConfig()]);
  @override double solve() {
    double x = x0;
    for (int i = 0; i < cfg.maxIter; i++) {
      double fx = f(x); track(fx.abs());
      if (fx.abs() < cfg.tolerance) return x;
      double d = df(x);
      if (d.abs() < 1e-12) throw StateError('derivative ~0 at $x');
      x = x - fx / d;
    }
    return x;
  }
}
// NewtonSolver((x)=>x*x-2,(x)=>2*x,1).solve() ≈ 1.414214 (faster!)
```

Math Solver 3 — Simpson Integrator

```
class SimpsonIntegrator extends Solver<double> {
  final double Function(double) f;
  final double a, b; final int n; // n even
  SimpsonIntegrator(this.f, this.a, this.b, [this.n = 100]);
  @override double solve() {
    if (n % 2 != 0) throw ArgumentError('n must be even');
    double h = (b - a) / n, s = f(a) + f(b);
    for (int i = 1; i < n; i++) {
      s += (i % 2 == 0 ? 2 : 4) * f(a + i * h);
    }
    return s * h / 3;
  }
}
// SimpsonIntegrator((x)=>x*x, 0, 1).solve() ≈ 0.3333 (= 1/3)
```

Which Solver When?

Problem	Pick	Why
Root, only sign change known	Bisection	Guaranteed, slow-linear
Root + derivative known	Newton	Quadratic, needs good x_0
Definite integral	Simpson	$O(h^4)$, n even
All	Behind `Solver<T>`	Swap in one line, test all

- **Discuss (1 min):** For x^2-2 on [0,2], which converges in fewer iterations? Why might you still ship Bisection in a student app?

Async in Flutter — FutureBuilder Teaser (→ W4-5)

- Console `await` becomes UI `FutureBuilder(future: ..., builder: ...)`
- States map 1:1: waiting → spinner, data → result card, error → message

```
FutureBuilder<double>(
  future: solveAsync(solver),
  builder: (ctx, snap) =>
    snap.hasData ? Text('root=${snap.data}') :
    snap.hasError ? Text('err: ${snap.error}') :
    CircularProgressIndicator(),
)
```

- Full wiring in Weeks 4-5. Today: master the `Future` itself.

✏️ Mini-Task — Design Before You Code (3 min)

- **Task:** Sketch a small shape API: which shared behavior belongs in an abstract `Shape`, and what should a `Drawable` interface require?
- Share with a neighbour: can different shapes be stored in one `List<Drawable>` and drawn with the same method call?

- Good API test: client code depends on the `Shape` / `Drawable` contract, not each concrete class
- Keep this sketch — the lab turns it into a working class hierarchy

🧪 Lab 1/6 — Abstract Shapes (10 min)

Create `01_abstract_shapes.dart` in DartPad or VS Code.

1. Declare `abstract class Shape` with `area()` and `perimeter()`.
2. Implement `Rectangle` and `Circle`, overriding both methods.
3. Create one of each and print their area and perimeter.

Done when: both concrete shapes provide working implementations of the full contract.

🧪 Lab 2/6 — Interfaces: Drawable (15 min)

Create `02_interfaces.dart`.

1. Declare `abstract class Drawable` with `draw()`.
2. Implement `Drawable` in `Circle` and `Square`.
3. Store both objects in a `List<Drawable>` and call `draw()` in a loop.
4. Explain why `implements` requires each class to provide `draw()`.

> - **Check:** The loop should work through the `Drawable` API without checking concrete types.

🧪 Lab 3/6 — Multiple Interfaces (10 min)

```dart
abstract class Flyable {
  void fly();
}
abstract class Swimmable {
  void swim();
}
class Duck implements Flyable, Swimmable {
  @override
  void fly() => print('Duck is flying');
  @override
  void swim() => print('Duck is swimming');
}
```

Create `03_multiple_interfaces.dart`, then instantiate `Duck` and call both methods.

Discuss: what compile-time error appears if either method is missing?

🧪 Lab 4/6 — Abstract Class + Interface (10 min)

Create `04_mixed_oop.dart` using the handout's `Animal` and `Pet` contracts.

1. `Animal` is abstract and requires `makeSound()` and a `species` getter.
2. `Pet` requires `play()`.
3. `Cat extends Animal implements Pet`; implement all three members.
4. Call the methods and read `species` from `main()`.

Discuss: Which implementation is inherited through `extends`, and which contract comes from `implements`?

🧪 Lab 5/6 — Challenge: ElectricCar (15 min)

Create `05_comprehensive_challenge.dart`.

1. Define abstract `Vehicle` with `brand`, `speed`, `accelerate()`, and `getTopSpeed()`.
2. Define an `ElectricVehicle` contract with `batteryLevel` and `charge()`.
3. Implement both in `ElectricCar extends Vehicle implements ElectricVehicle`.
4. In `main()`, print the car, accelerate it, show its top speed, then charge it.
5. Observe that `implements` requires `ElectricCar` to supply every member of the interface.

> - **Stretch:** Store one or more electric vehicles through the `ElectricVehicle` type and call `charge()` without relying on `ElectricCar`.

🧪 Lab 6/6 — Acceptance + Submission

- [ ] `01_abstract_shapes.dart`: Rectangle and Circle calculate area and perimeter
- [ ] `02_interfaces.dart`: a `List<Drawable>` can draw each shape
- [ ] `03_multiple_interfaces.dart`: Duck implements and uses both interfaces
- [ ] `04_mixed_oop.dart`: Cat extends Animal and implements Pet
- [ ] `05_comprehensive_challenge.dart`: ElectricCar implements the required behavior
- Run `dart format` and `dart analyze`, then paste output for exercises 1–4 into the group chat.
- Bonus: complete and submit the ElectricCar challenge.

⚠️ Common Mistakes

- Forgetting `super` params → red screen; match parent constructor shape
- `_private` assumed file-proof — it's **library**-private, not class-private
- `implements` does not inherit implementation — re-implement every required member
- `==` without `hashCode` → `Set` dedupe silently fails
- `await` in non-`async` function; `Future.wait` one failure kills all
- Forgetting a getter or setter in an implemented interface leaves the class incomplete

Resources

- Dart Language Tour: **dart.dev/guides/language/language-tour** (mixins, generics, async)
- Effective Dart: **dart.dev/effective-dart** (naming `_`, equality rules)
- Async codelab: **dart.dev/codelabs/async-await** + `Future.wait` docs
- Lab fallback: **DartPad — dartpad.dev** (no install, share link)
- Book: *Dart Apprentice* ch. 9-11 (OOP II) + ch. 13 (Futures)

🏠 Homework (Due Week 4)

1. Finish the ElectricCar challenge if not completed in class
2. Read Dart Tour sections: **Mixins, Generics, Asynchrony** (30 min)
3. Install check: `flutter doctor` clean-ish + Counter on Chrome (W1 carry-over)
4. **Teams FINAL:** if not submitted today, post team + repo link in course channel by Friday
5. Bring **one** async question (e.g. "when Future vs Stream?") for W4 warm-up

> - **Recap hint:** Next week's 5-min quiz picks from *this* deck — likely `implements vs with` or `await` semantics. Annotate one slide tonight.

Next Week — Week 4 Preview

- **Widgets & Layouts:** Material, `Row/Column/Stack`, `ListView`, Forms + validation
- Lab: calculator / function-plotter UI — display computed results in a Flutter layout
- Pre-read: `StatelessWidget` vs `StatefulWidget` + `setState` (15 min)
- Keep Dart OOP files — the next weeks build on these class and interface patterns

Questions & Discussion

Thank you!

Abstract contracts, implement interfaces, and never block the UI thread!